TECHNICAL PROJECT REPORT

GlobalWatch

World Air Quality Intelligence Platform — a Lambda architecture on Microsoft Fabric spanning real-time streaming, a batch medallion lakehouse, machine learning, and an LLM-powered analytics interface.

Author	Jayanth Dolai
Platform	Microsoft Fabric — Trial capacity, Central India
Repository	github.com/demonjd2026-afk/globalwatch-fabric
Live app	globalwatch-fabric.streamlit.app
Data snapshot	09 August 2026
Report version	2.0

894	303	23	274	5,056
READINGS	STATIONS	COUNTRIES	WHO BREACHES	EVENTS / RUN

Figures reflect the Gold layer as published on 09 Aug 2026 and grow with every pipeline run.

1. Executive summary

GlobalWatch is an end-to-end air quality data platform built on Microsoft Fabric. It ingests live sensor readings from the OpenAQ v3 API across 70 targeted country codes, processes them through a Bronze → Silver → Gold medallion lakehouse in PySpark, scores PM2.5 readings with a Spark MLlib model tracked in MLflow, streams the same source into a KQL Eventhouse for sub-minute alerting, and serves the result through a Direct Lake Power BI model and a public Streamlit application with a Claude-powered analytics assistant.

The platform runs on a schedule rather than on demand: a five-notebook batch pipeline daily at 02:00 IST, and an hourly streaming producer. Every component described in this report has been executed and evidenced by screenshots in the repository; where a capability is blocked by tenant or SKU limits, this report says so explicitly rather than describing intent as delivery.

894	30.6%	96.15%	8 / 8	39
FACT ROWS	WHO EXCEEDANCE RATE	MODEL ACCURACY	SPARK PATTERNS	EVIDENCE SCREENSHOTS

What the data shows

Of 894 readings across 23 countries, **274 (30.6%) breach the WHO 2021 annual guideline for their pollutant**. The distribution is highly uneven: every Indian and Mongolian PM2.5 reading in the snapshot exceeds the guideline, India by a factor of 11.5 and Mongolia by 8.8, while Belgium, Hungary, the Netherlands and the United Kingdom all sit comfortably below it. The single worst reading is 300 µg/m³ at Delhi Technological University — twenty times the WHO guideline, and squarely in the Hazardous band.

What was built

Capability	Delivered
Medallion lakehouse (Bronze → Silver → Gold) in PySpark	**Built**
SCD Type 2 dimension via Delta MERGE, V-Order and Z-Order on Gold	**Built**
Eventstream → KQL Eventhouse with transactional update policy	**Built**
Data Activator hazard alerting (email confirmed firing)	**Built**
Random Forest + MLflow registry, scoring written back to Gold	**Built**
Batch and real-time Data Factory pipelines on schedule	**Built**
Direct Lake semantic model with continent-level RLS + report	**Built**
Public Streamlit dashboard with grounded Claude assistant	**Built**
Native Fabric Data Agent	**Blocked — F64+ SKU required**
Fabric Git integration / deployment pipelines	**Blocked — trial tenant**
WAQI, World Bank and OpenMeteo enrichment	**Designed**
Tool-use agent against live KQL / SQL endpoints	**Designed**

2. Problem and objectives

Air quality is published by dozens of independent monitoring networks through incompatible APIs, at different cadences, with different units and no shared classification. A city planner asking "which of my stations breached the WHO guideline this month, and is it getting worse" has no single place to look; an operations team wanting to be told when a station goes hazardous has to build the detection themselves.

GlobalWatch set out to close that gap with five objectives:

1. **Unify** — bring readings from many countries into one conformed model with consistent units and one WHO-based classification applied identically in the batch and real-time layers.
2. **Historise** — track station identity over time so a reading can always be attributed to the station configuration that produced it.
3. **React** — detect hazardous conditions in seconds, not on the next daily run, and act on them automatically.
4. **Predict** — score readings with a tracked, registered, reproducible model rather than a notebook artefact.
5. **Open up** — make the result usable by a BI analyst, an ad-hoc SQL user, and someone who only wants to ask a question in English.

> **Design constraint that shaped everything:** the OpenAQ free tier allows 60 requests per minute. Reaching 70 countries inside that budget drove two decisions — use `/locations/{id}/latest` (one call per location rather than one per sensor) and bound concurrency with a semaphore rather than serialising with sleeps. Together these turned a handful of countries into 23 with live sensor coverage.

3. Architecture

The platform is a Lambda architecture: a batch path for historical analytics and machine learning, and a speed path for operational awareness and alerting. Both operate over the same OneLake, so there is no copy-based integration between them — the engines read the same Delta files.

3.1 Two paths, one storage layer

Path	Orchestration	Cadence	Latency	Serves
Batch	`pl_batch_globalwatch` — 5 chained notebooks	Daily 02:00 IST	Hours	Power BI, ML scoring, published snapshot
Real-time	`pl_realtime_globalwatch` — `Stream_To_Eventstream`	Hourly	Seconds	KQL queryset, hazard alerts, live counter

3.2 Key architecture decisions

Microsoft Fabric over assembled Azure services

One platform rather than ADF + Databricks + Synapse + Power BI. OneLake removes copy-based silos — Spark, T-SQL and KQL read the same Delta files. Eventstream, Eventhouse and Data Activator are first-party, so there are no custom connectors to maintain. Direct Lake removes the import/refresh cycle entirely. *Trade-off:* Fabric is newer, and some capabilities are gated by SKU size — this project hit exactly that with the Data Agent.

Lambda over Kappa

The batch layer needs multi-pass work that streaming does not suit: SCD2 maintenance, star schema joins, model training and scoring. The speed layer needs seconds-level freshness that Delta streaming cannot reach. *Trade-off:* two code paths — mitigated because the speed path is declarative, with the only custom code being the producer notebook.

KQL Eventhouse for the speed layer

KQL is built for time-series, and update policies apply stream-time transformation with no separate compute. Setting `IsTransactional: true` means raw and silver cannot diverge — if the transform fails, the raw write rolls back. Data Activator binds natively with no connector. *Trade-off:* two query languages, accepted because the surface area actually needed is small.

Publish Gold snapshots to GitHub rather than expose an endpoint

The pipeline writes JSONL extracts of Gold into the repository; the public app reads them over `raw.githubusercontent.com` with a 30-minute cache. The app costs nothing to host, needs no inbound path into the Fabric workspace, holds no credential beyond the LLM key, and every published number carries a versioned, auditable commit. *Trade-off:* the app shows the last export rather than live Gold, bounded by pipeline cadence.

4. Data model

`fact_readings` is deliberately a **long** fact — one row per station × pollutant × reading, with a `parameter` column and a `value` column, rather than one column per pollutant. Adding a sixth pollutant therefore requires no schema change, and `dim_pollutant` can own the WHO guideline per parameter instead of thresholds being hard-coded into the fact.

Table	SCD	Rows	Key characteristics
`fact_readings`	—	894	V-Order; Z-Order on (location_id, reading_ts); partitioned by year_month
`dim_station`	Type 2	308	Delta MERGE; active_flag, effective_start/end, MD5 station_hash
`dim_country`	Type 1	23	Continent mapping; World Bank enrichment columns reserved
`dim_date`	Type 0	5,844	Pre-generated spine 2015-01-01 → 2030-12-31
`dim_pollutant`	Type 0	5	WHO 2021 annual guideline values
`fact_aqi_predictions`	—	245	Random Forest output over 206 stations; overwritten each run

4.1 SCD Type 2 on stations

Stations are renamed and reassigned between districts, and regulatory reporting needs to know which station configuration produced a given reading. The MERGE runs in two steps within one transaction:

```
Step 1 — expire changed rows
  MATCH  target.location_id  = source.location_id
     AND target.active_flag  = true
     AND target.station_hash <> source.station_hash
  SET    active_flag = false, effective_end = current_timestamp()

Step 2 — insert new versions
  Anti-join source against the active target set
  INSERT with active_flag = true, effective_end = 9999-12-31
```

The `station_hash` is an MD5 of location_id, name, city and country. Comparing one hash rather than every attribute keeps change detection O(1) in table width — adding a column to the dimension does not make the comparison more expensive.

4.2 Surrogate keys

All surrogate keys are CRC32 hashes of the natural key, cast to integer. This is deterministic across runs and executors, so re-running a load is idempotent and needs no distributed counter or coordination shuffle. The trade-off is that keys carry no insertion order — which is fine, because ordering is expressed by `effective_start` and `date_key`.

4.3 Data quality rules (Silver)

#	Rule	Filter	Purpose
1	Empty parameter	`parameter != ""`	Drop unclassified sensors
2	Known pollutants	`parameter IN (pm25, pm10, no2, co, o3)`	Keep criteria pollutants only
3	Valid range	`0 < value < 10000`	Drop sensor malfunctions
4	Deduplication	`dropDuplicates(location_id, parameter, reading_ts)`	Remove repeated API records

4.4 WHO guidelines applied

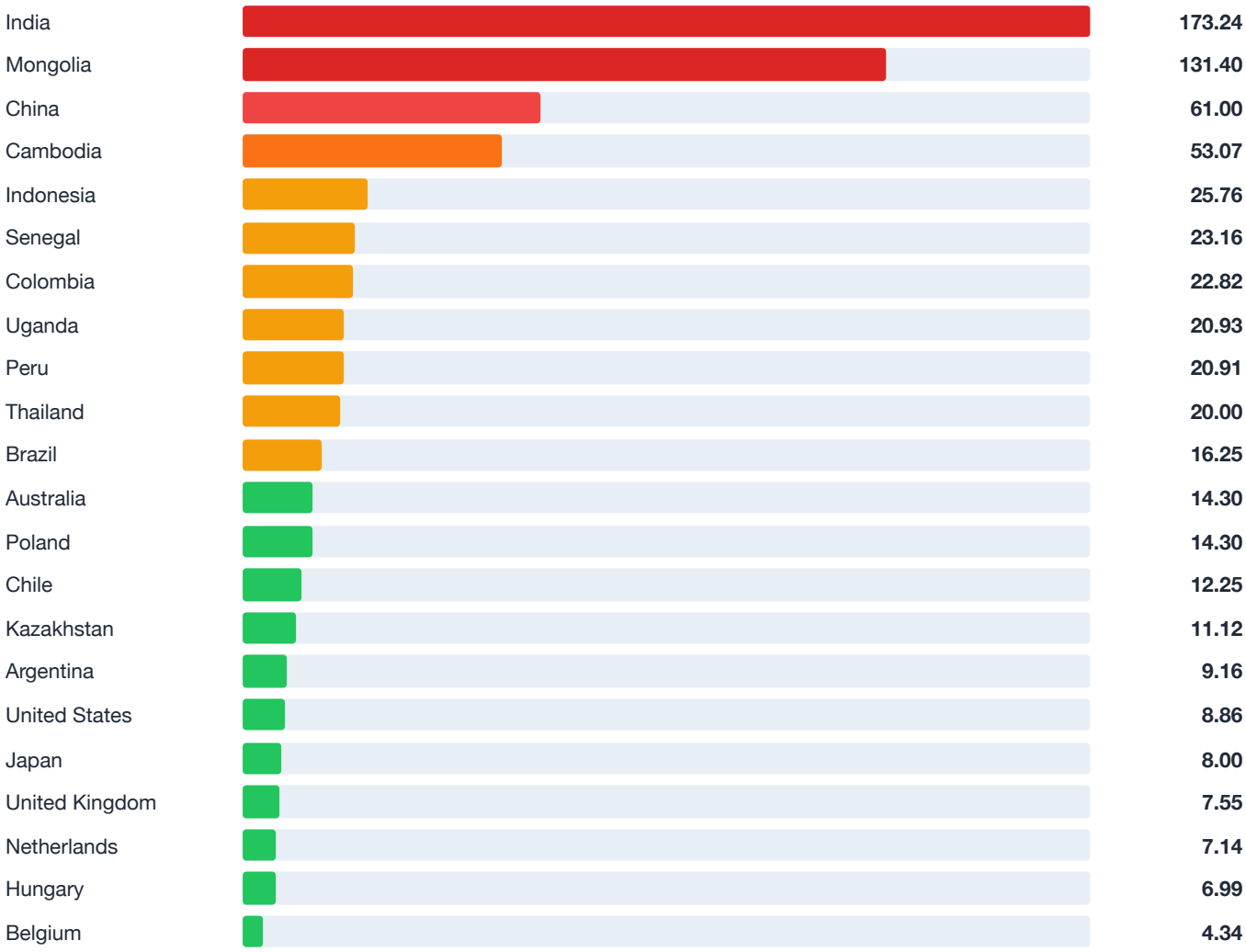
Pollutant	WHO 2021 annual guideline	Unit	Readings held
PM2.5	15.0	µg/m³	245
PM10	45.0	µg/m³	189
NO_2	10.0	µg/m³	201
CO	4000.0	µg/m³	91
O_3	60.0	µg/m³	168

AQI categorisation is applied to PM2.5 only — the other four pollutants carry `N/A`, which is why 649 of 894 rows show that value. The WHO *exceedance flag*, by contrast, is applied to every pollutant using its own guideline.

5. Findings from the current data

5.1 Average PM2.5 by country

The dashed WHO annual guideline sits at 15 µg/m³. Bars are scaled against the highest value in the set.



Green = at or below the WHO guideline of 15 µg/m³ · amber = 1–2× · orange/red = above 2×. Values in µg/m³.

5.2 Largest guideline breaches by country and pollutant

Country	Pollutant	Average	WHO limit	Factor	Readings	% over
India	PM2.5	173.24	15	11.5×	14	100%
Mongolia	PM2.5	131.40	15	8.8×	5	100%
India	NO_2	75.24	10	7.5×	17	100%
India	PM10	279.97	45	6.2×	9	100%
China	NO_2	61.83	10	6.2×	6	100%
Mongolia	PM10	230.62	45	5.1×	8	88%
Brazil	NO_2	47.73	10	4.8×	11	100%
China	PM2.5	61.00	15	4.1×	15	87%
Mongolia	NO_2	36.31	10	3.6×	14	100%
Peru	PM10	131.34	45	2.9×	10	100%
Chile	NO_2	25.19	10	2.5×	7	86%
United Kingdom	NO_2	15.99	10	1.6×	37	57%

5.3 Most polluted stations by peak PM2.5

Station	Country	Peak PM2.5	AQI band
Delhi Technological University	India	300.0	Hazardous
Income Tax Office, Delhi — CPCB	India	219.0	Hazardous
Bukhiin urguu	Mongolia	207.0	Hazardous
R K Puram, Delhi — DPCC	India	195.0	Hazardous
Chizhou Lao Gan Bu Ju	China	175.0	Hazardous
MNB	Mongolia	172.0	Hazardous
Amgalan	Mongolia	134.0	Unhealthy

5.4 Distributions

AQI category (PM2.5 only)	Readings	Continent	Readings	Pollutant	Readings
N/A (non-PM2.5)	649	Europe	314	PM2.5	245
Good	137	South America	192	NO_2	201
Moderate	72	Asia	181	PM10	189
Unhealthy	17	Other	89	O_3	168
Hazardous	11	North America	67	CO	91
Unhealthy for Sensitive	8	Africa / Oceania	42 / 9	—	—

Read these figures with their sample sizes. India's 11.5× exceedance rests on 14 PM2.5 readings and Mongolia's on 5. The direction is unambiguous and consistent with published research, but the platform currently holds a broad, shallow sample — the `/latest` endpoint gives one reading per sensor per run. Deepening the history for leading countries is the top item on the roadmap in §10.

6. Engineering detail

6.1 Spark optimisation patterns

All eight patterns are implemented in the notebooks, each chosen for a specific characteristic of this workload rather than applied generically.

Pattern	Where	Why here
AQE — enabled, coalescePartitions, skewJoin	All notebooks	Runtime re-planning; without coalescing, dimension joins leave ~200 tiny shuffle files
Broadcast join	Gold fact build	Every dimension is tiny (23 / 5 / 308 rows); explicit hints hold even when AQE statistics are stale
Salting	Silver station joins	Dense-city stations return far more readings than the median; salting prevents a single-executor straggler
Partitioning	Silver, Gold	country_code + year_month in Silver, year_month in Gold — matches the dominant filter pattern
Z-Order	fact_readings	Reports filter station and time together; Data Skipping eliminates non-matching files
V-Order	All Gold writes	Mandatory for Direct Lake — without it Power BI falls back to a slower scan mode
Delta MERGE	dim_station SCD2	Atomic expire + insert; two separate statements would risk a partial state
mergeSchema	Bronze writes	OpenAQ adds fields occasionally; the write extends the schema rather than failing

6.2 Real-time layer

The Eventstream custom endpoint receives readings from the producer notebook and routes them to `raw_readings` in the KQL Eventhouse. An update policy fires `TransformRawReadings()` on every ingestion, writing the enriched row into `silver_readings` in the same transaction:

```
.create-or-alter function TransformRawReadings() {
    raw_readings
    | where parameter in ("pm25", "pm10", "no2", "co", "o3")
    | where value >= 0 and value < 10000
    | extend aqi_category = case(
        parameter == "pm25" and value <= 12.0,  "Good",
        parameter == "pm25" and value <= 35.4,  "Moderate",
        parameter == "pm25" and value <= 55.4,  "Unhealthy for Sensitive",
        parameter == "pm25" and value <= 150.4, "Unhealthy",
        parameter == "pm25" and value >  150.4, "Hazardous",
        "N/A")
    | extend exceeds_who_guideline = case(
        parameter == "pm25" and value > 15.0,   true,
        parameter == "pm10" and value > 45.0,   true,
        parameter == "no2"  and value > 10.0,   true,
        parameter == "co"   and value > 4000.0, true,
        parameter == "o3"   and value > 60.0,   true,
        false)
    | project location_id, location_name, city, country_code, country_name,
              latitude, longitude, parameter, value, unit,
              aqi_category, exceeds_who_guideline,
              reading_ts, ingestion_ts, source_system
}
```

Retention is 30 days on raw and 365 on silver. Because `IsTransactional` is true, a transform failure rolls back the raw write — the two tables cannot diverge. Crucially, the thresholds here are identical to those applied in the Silver notebook, so the batch and speed layers can never disagree about whether a reading breached a guideline.

Alerting

Data Activator monitors the `pm25_station` object on the same Eventstream and fires when PM2.5 exceeds 150 µg/m³ — the WHO Hazardous threshold. The action is an email carrying station name, city, country and the current reading. The rule is running against 98 monitored station IDs with 5 actively triggering; the alert email was confirmed received on 09 Aug 2026 at 06:08 UTC.

6.3 Machine learning

Property	Value
Algorithm	Spark MLlib `RandomForestClassifier`, numTrees=100, maxDepth=5
Features	pm25, pm10, no2, o3, co — pivoted long → wide, nulls filled with 0
Label	5-class WHO PM2.5 banding, 0 = Good … 4 = Hazardous
Split	80/20, seed=42
Tracking / registry	MLflow — params, metrics, artifact; registered as `globalwatch_aqi_classifier` v1
Test accuracy	96.15%
Feature importance	PM2.5 76.32%, PM10 12.91%, remainder across NO_2 / O_3 / CO
Inference output	`fact_aqi_predictions` — 245 rows, 206 stations; Good 130, Moderate 87, Unhealthy 20, Hazardous 8

Random Forest was chosen for tolerance of class imbalance on a small dataset, no feature-scaling requirement across very different pollutant ranges, native Gini feature importance, and availability in Spark MLlib without installing anything.

> **An honest reading of 96.15%.** PM2.5 is by construction the dominant signal for a PM2.5-derived label, so the headline number demonstrates a working end-to-end MLflow lifecycle far more than it demonstrates a hard prediction problem. The transferable value is the pattern — track, register, load by URI, score, persist idempotently — which applies unchanged on Databricks or EMR. Making this genuinely predictive would mean forecasting *future* readings from weather and traffic features, which is the model-improvement item in §10.

6.4 Orchestration

Pipeline	Activities	Schedule	Resilience
`pl_batch_globalwatch`	Bronze → Silver → Gold → ML → Data Agent simulation	Daily 02:00 IST	Retry 2, interval 60s, timeout 1h, email on failure, on-success dependencies only
`pl_realtime_globalwatch`	`Stream_To_Eventstream`	Hourly	Retry 3, interval 30s, timeout 30 min

Dependencies are strictly on-success: a Silver failure must not allow a stale Gold to be published to the semantic model. The latest real-time run succeeded in 1m 44s and dispatched 5,056 events.

7. Serving and consumption

Consumer	Path	Notes
BI analyst	Direct Lake semantic model over Gold	Four active relationships; `ContinentViewer` RLS role on dim_country; two-page report
Ad-hoc SQL user	`globalwatch_warehouse`	Cross-lakehouse T-SQL; the Warehouse cannot write Delta, so it is not a medallion target
Operations	KQL queryset + Data Activator	Seven dashboard queries; email alert above 150 µg/m³
Public / non-technical	Streamlit Cloud reading published JSONL	Dashboard plus grounded Claude assistant

7.1 The Streamlit application

The dashboard page carries six KPI tiles, a global PM2.5 bubble map, average PM2.5 by country against the WHO line, an AQI donut, the ML prediction distribution, and a top-ten polluted stations table. It reads the published snapshots from GitHub with a 30-minute cache — there is no live connection to Fabric.

7.2 The natural language assistant

The assistant answers questions in English over the same data. Its design decision is **grounding**: rather than giving the model query access, the app pre-computes aggregates from the loaded snapshot — per-country PM2.5 averages, WHO exceedance counts, AQI distribution, ML prediction distribution, top polluted stations, an architecture summary and the WHO guideline — and injects them as the system prompt on every turn.

Property	Value
Endpoint	`POST https://api.anthropic.com/v1/messages`
Model	`claude-sonnet-4-6`
max_tokens	1000
Auth	`x-api-key` from Streamlit secrets — never committed
Conversation	Full history sent each turn (the Messages API is stateless)

> **What this is, and what it is not.** This is a single grounded API call per turn, not a multi-tool agent. The benefit is that every figure the assistant states is traceable to the published snapshot and cannot be hallucinated; the limitation is that it cannot answer questions outside those aggregates. A three-tool design — `query_kql` for live conditions, `query_gold_sql` for historical aggregates, `get_country_health_context` for economic enrichment — is fully specified in the technical specification as the next step once the agent has network access to the KQL and Lakehouse SQL endpoints.

8. Constraints encountered and how they were handled

Constraint	Impact	Resolution
Fabric Data Agent requires an F64+ SKU	No managed natural-language querying on trial capacity	NL → SQL pattern demonstrated explicitly in a notebook; user-facing NL delivered by the Streamlit assistant instead
Git integration blocked by tenant account type	Fabric items are not automatically version-controlled	This repository is the source of truth; configuration documented in the setup guide and evidenced by 39 screenshots
`%pip` magic disabled in pipeline runs	The streaming notebook succeeded interactively but failed under the pipeline	`azure-eventhub` moved into the `globalwatch-env` environment as a PyPI dependency
Outbound KQL calls blocked in the trial pipeline sandbox	The KQL verification cell failed under the pipeline	Cell short-circuits in pipeline mode with an explanatory message; run interactively to verify counts
OpenAQ free tier — 60 requests per minute	Naive serial ingestion could not cover 70 countries	Semaphore-bounded 10-worker parallel fetch, plus `/latest` for one call per location rather than per sensor

> **The most transferable lesson.** Interactive notebook execution and pipeline execution are different environments with different constraints — library management and network egress both behave differently. Two of the five constraints above only appeared once the notebooks ran under Data Factory rather than interactively, which is a strong argument for validating in pipeline mode before treating a pipeline as finished.

9. Cost and security

9.1 Cost model

Component	Basis	Monthly estimate
Fabric F2 capacity	~$0.36/hr, ~4 active hrs/day, paused otherwise	~$43
OneLake storage	~$0.023/GB; dataset well under 1 GB	< $1
Eventstream, Eventhouse, Data Activator	Included in capacity	$0
Streamlit Community Cloud	Free tier	$0
OpenAQ / WAQI / World Bank / OpenMeteo	Free tiers	$0
Anthropic API	Usage-based; low-volume assistant traffic	Low single digits
Total		**≈ $45**

Controls in use: pause the capacity immediately after each session, run OPTIMIZE and VACUUM on a schedule, use Starter Pools rather than custom Spark pools (no idle compute charge), and keep the streaming producer hourly rather than continuous.

9.2 Security posture

Secret	Storage
OpenAQ API key	`globalwatch-env` Spark property
Event Hub connection string	`globalwatch-env` Spark property
GitHub PAT (scoped to contents write)	`globalwatch-env` Spark property
Anthropic API key	Streamlit Cloud secrets

`.gitignore` excludes `.env`, `secrets.json`, `credentials.json` and `.streamlit/secrets.toml`. No credential appears anywhere in committed code. Row-level security is enforced by a `ContinentViewer` role on `dim_country`, validated with Power BI's View-as-role. Gold tables are labelled as public data — air quality readings are public information — while connection strings remain environment-scoped only.

10. Roadmap

Ordered by the value each would add if this moved beyond a portfolio build:

1. **Historical depth.** Replace `/latest` with a windowed measurement pull for the leading countries, so trend analysis has depth as well as breadth and the sample-size caveat in §5 goes away.
2. **CI/CD.** Azure DevOps Git integration plus Dev → Test → Prod deployment pipelines, with promotion gated on Silver DQ results and Gold assertion counts.
3. **Enrichment.** Deploy the World Bank and OpenMeteo Dataflows to populate the reserved `dim_country` columns and add weather features to the model — the schema already accommodates both, so the change is additive.
4. **Live-endpoint agent.** Implement the three-tool design so natural-language answers reach beyond the pre-computed aggregates.
5. **Data quality as a first-class artefact.** Persist DQ rule outcomes to a `dq_results` Delta table and alert on rule regressions, rather than only on pipeline failure.
6. **Model improvement.** Reframe as forecasting rather than classification of a derived label; add class weighting for the minority Hazardous class and scheduled retraining with version comparison in MLflow.
7. **Observability.** A `pipeline_run_log` Delta table capturing per-activity row counts and durations, surfaced as an operations page in the report.

11. Appendix — repository and evidence

11.1 Repository contents

Path	Contents
`README.md`	Overview, architecture, current findings, screenshot index
`SETUP.md`	Phase-by-phase reproducible build guide with verification checklist
`TECH_SPEC.md`	ADRs, schemas, pipeline and AI specifications, cost model, security
`notebooks/`	Seven PySpark notebooks plus a cell-by-cell guide
`kql/`	Eventhouse DDL (tables, transform function, update policy, retention) and seven dashboard queries
`streamlit/`	Public app, requirements, and the five published Gold snapshots
`docs/`	Architecture, data model, Spark patterns, interview narratives, this report
`screenshots/`	39 screenshots evidencing every phase

11.2 Evidence by phase

Phase	Screenshots
Workspace and storage	01 Fabric home · 02 workspace · 03 three lakehouses · 04 warehouse
Medallion pipeline	10 Bronze run · 11 Silver validation · 12 Spark UI (AQE) · 13 Gold SCD2 · 14 Delta detail · 35 post-pipeline counts
Real-time intelligence	16 KQL database · 17 Eventstream configured · 18 live data · 33 raw_readings count
Alerting	20 Activator rule running · 21 hazard alert email received
Machine learning	25 predicted classes · 26 MLflow run · 27 predictions written to Gold
Natural language	28 NL→SQL pattern · 29 query results · 27–29 assistant answers
Orchestration	31 batch scheduled · 34 batch succeeded · 32 realtime scheduled and succeeded · 30 Git limitation
Serving	22 semantic model relationships · 23 RLS role · 24 report page 1 · 36 report page 2 · 37 Streamlit dashboard · 38 Streamlit assistant

GlobalWatch — World Air Quality Intelligence Platform. Built by Jayanth Dolai on Microsoft Fabric. Repository: github.com/demonjd2026-afk/globalwatch-fabric · Live application: globalwatch-fabric.streamlit.app · Data snapshot 09 August 2026. All figures in this report were computed directly from the published Gold-layer snapshots in `streamlit/data/` and will change with each pipeline run.